



# CHAPTER 1: CLIENT-SIDE TECHNOLOGY STACK

Remark: This chapter is intended to bring back to mind competencies that you acquired in a previous lecture.

# LEARNING OBJECTIVES

- Understand basic software architectures for web applications
- Be able to develop HTML pages
- Be able to use CSS to style your HTML pages
- Be able to use JavaScript to handle interactions (local and remote) and logic on the web client side



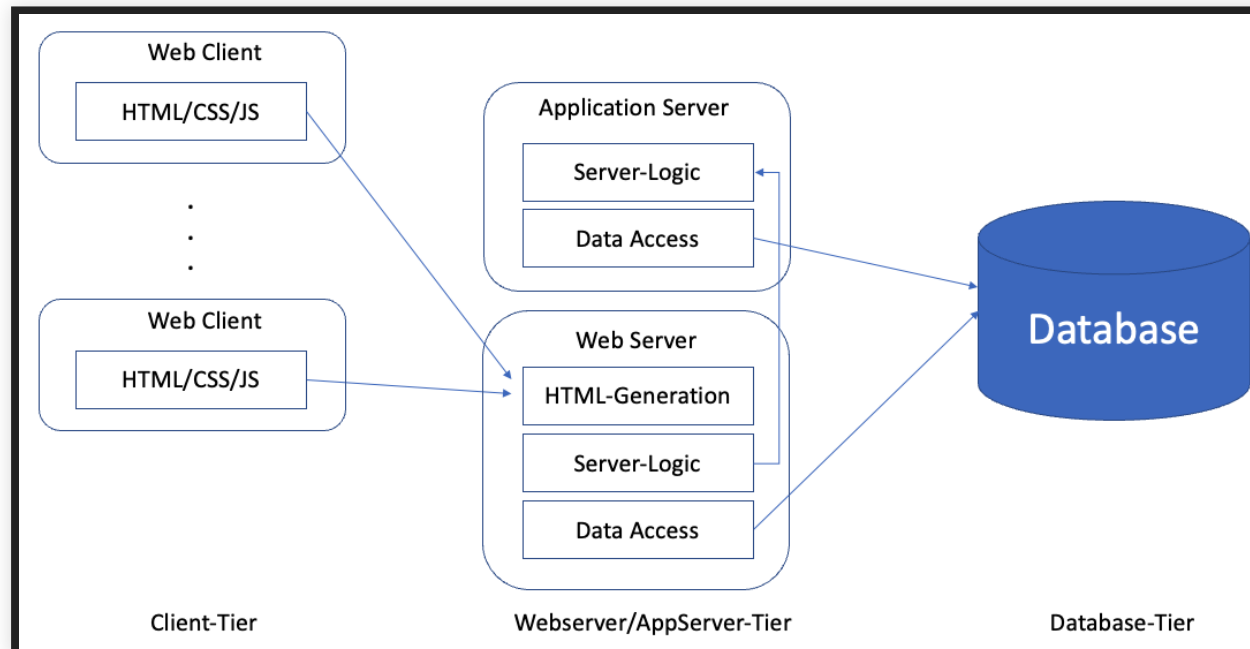
# 1.1 INTRODUCTION

A web application is always some kind of client/server system with a web browser being the client. Modern web browsers incorporate interpreters for HTML5:

$$HTML5 = HTML + CSS + JavaScript$$

In this section we will explain how modern web applications work in terms of client-server interactions. While the client side is typically a web browser, there are some variations on the server side.

# WEBCIENT, WEBSERVER[, APPLICATION SERVER] AND DATABASE (3/4-TIER)



- The "thinnest" type of client is a web browser which causes no application installation effort
- The browser sends a HTTP request to the web server, which in turn uses a database or an application server to generate a HTML reply

## **Terms: Web Server, Application Server (AS)**

### **Web Server (when AS is present):**

- handles static content (HTML, CSS, JavaScript)
- may handle dynamic content or leave it up to an AS
- does not provides business logic or perform database operations

### **Application Server (in context of web server):**

- may serve dynamic web content
- provides business logic
- performs database accesses
- offers application level services: transaction management, authorization, load balancing, caching, ...

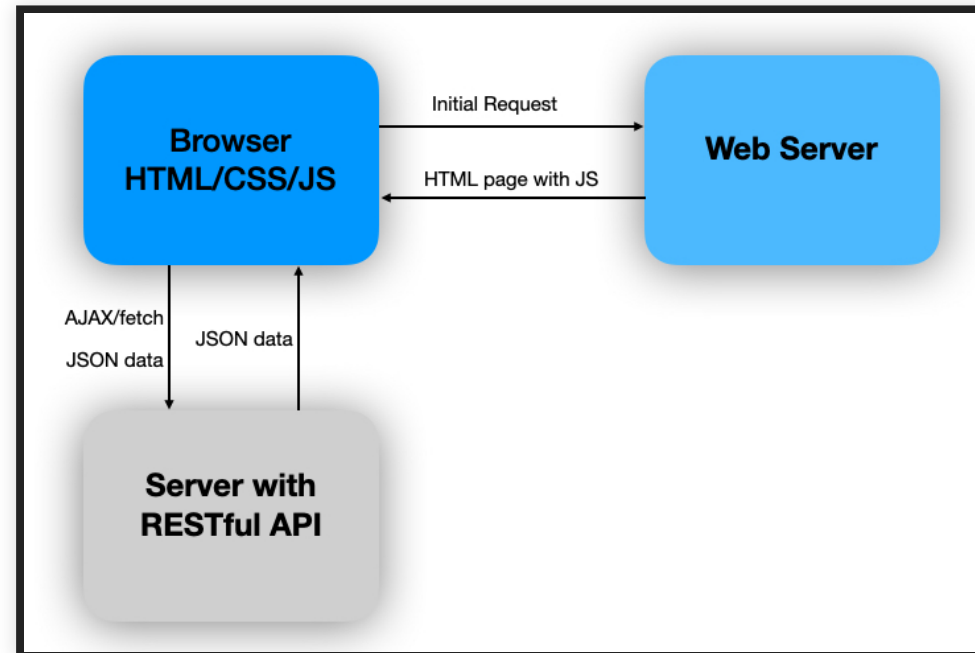
### Use when...

- your app has many users
- user interface requirements can be realized in browser

### Disadvantages

- Browser application with limitations in terms of look & feel and user experience (UX)
- Web page generation on the server side limits interactivity and UX due to page reloading

# SINGLE PAGE APPLICATION (SPA)



- On the first request the browser loads the entire application code from the web server
- As the user interacts with the app the REST server is used to load or save data in JSON format

## Advantages

- Can handle apps with many users (scales well due to REST approach)
- Reduces server load by minimising server calls, in particular no page generation on the server (see PHP)

## Disadvantages

Browser application with limitations in terms of look & feel and user experience (UX)



## 1.2 HTML

In this section we will be referring to the hypertext markup language (HTML) which is part of HTML5. HTML5 encompasses HTML, CSS, JavaScript and other specifications.

**HTML (hypertext markup language)** ist a language for describing so-called hypermedia documents which are used in the WWW.

A markup - also called **tag** - represents a control element for formatting and placing content (text, images, graphics, video, audio, etc.) within a hypermedia document.

## TAG SYNTAX

```
<tag  
  attribute1="value1" attribute2="value2">  
  <!-- text and or other tags -->  
</tag>
```

- a tag typically has a closing tag (or not: `<input ...>` or `<br />`)
- a tag may have attributes
- this is a comment (multiline)

# HTML DOCUMENT STRUCTURE BY EXAMPLE

The following example shows the general document structure:

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello, World!</title>
</head>
<body>
  <h1>This is a header 1</h1>
  <hr />
  <p>This is some Text!</p>
  <p>
    <a href='http://www.thi.de' target='_blank'>
      This is a link
    </a>
  </p>
</body>
</html>
```

- DOCTYPE determines the underlying grammar for parsing the document, here: HTML5
- "html" is the top node of the document tree
- under "html" we find two child nodes: "head" and "body"
- "body" typically contains the document's contents

# IMPORTANT <HEAD> TAGS

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Cheat Sheet</title>
  <meta name="description" content="A brief page description">
  <link rel="stylesheet" href="/styles.css">
  <script src="/script.js"></script>
  <meta http-equiv="expires" content="01 Dec 2026 00:00:00 GMT">
</head>
```

- selects "utf-8" as character set for the document
- viewport setting for a responsive page, prevents the page to be scaled down on smartphones
- sets the title of the document
- sets meta information about the page, e.g. for web crawlers
- loads CSS stylesheet "styles.css"
- loads JavaScript file "script.js"
- controls how long the document can be cached by the browser

# IMPORTANT TAGS FOR DOCUMENT CONTENT

There is a number of important tags that you will likely be using in your future web pages. Here we will only list a few. Since we are focussing on application development, [semantic elements](#) will not be discussed. Please refer to your favorite resources to get detailed information on how to use these tags!

- Heading: `<h1>` - `<h6>`
- Paragraph: `<p>`
- List
  - `<ul>` - `<li>`: unordered lists
  - `<ol>` - `<li>`: ordered lists
  - change the list style type [using CSS](#)
- Hyperlink: `<a>`
  - variants: open in new browser tab, with embedded image/button
- Table: `<table>` `<caption>` `<thead>` `<th>` `<tbody>` `<tr>` `<td>` `<tfoot>`
- Image: `<img>`

(continued)

- rectangle for content: `<div>` (block mode), `<span>` (inline mode)
- horizontal ruler: `<hr>`
- Group content visually: `<fieldset>` `<legend>`
- Form: `<form>`
  - Label before input: `<label>`
  - Textfield: `<input>` or `<textarea>`
  - Dropdown: `<select>` `<option>`
  - Checkbox: `<input type="checkbox">`
  - Radio: `<input type="radio">`
  - Button: `<button>`

[W3Schools' HTML Reference](#) gives a full overview of html input types.

# EXERCISE: PERSON DATA ENTRY

A form for entering personal data is to be developed. The form should look like this:

Personendaten eingeben

Name:

Über mich

Essen: ☐ Fastfood ☒ Vegan

Geschlecht: ☐ weiblich  
☐ männlich  
☒ divers

Nationalität:

Use only HTML tags - don't use CSS styling yet!



# 1.3 CSS



# INTRODUCTION

CSS (Cascading Style Sheets) is a language used to control the appearance of HTML elements on a webpage. It allows you to apply styles like colors, fonts, spacing and layout to your HTML content.

# CSS SYNTAX

A CSS rule consists of one or more selectors and a declaration block:

```
selector1, selector2, ... {  
  property1: value;  
  property2: value;  
  ...  
}
```

- each selector determines the HTML elements that the properties should be applied to
- each line in the declaration block contains a property and the value it should be set to

## Simple examples:

```
/* set text color in all paragraphs to blue */  
p {  
    color: blue;  
}  
  
/* all borders in all tables should collapse */  
table, tr, td {  
    border-collapse: collapse;  
}  
  
/* all divs have blue background and a font size of 20px */  
div {  
    background-color: blue;  
    font-size: 20px;  
}
```

The above examples show how **element selectors** can be specified.

## CSS-SELECTORS - ID-SELECTOR

The following example shows the use of the Id selector.

```
<h1 id='top-secret'>  
  Top Secret Information  
</h1>  
<p>Cupcake ipsum dolor sit amet cake croissant fruitcake muffin. Sweet bonbo
```

- text color will be orange for all elements with id "top-secret"
- h1 will be selected

## CSS-SELECTORS - CLASS SELECTOR

The following example shows the use of the class selector.

```
<div class='yellow'>  
  This is some random text which also contains an  
  <span class='important'>important part (this text)</span>  
</div>
```

- classes can be valid for certain tags (here: DIV)
- ... or for all tags
- rule 1 is applied here
- rule 2 is applied here



# CSS-SELECTORS - ATTRIBUTE SELECTOR



# CSS-SELECTORS - CONTEXT SELECTORS (AKA COMBINATORS)



# CSS-SELECTORS - PSEUDO CLASSES



# CSS-SELECTORS - PSEUDO ELEMENTS

## Differences between Pseudo-classes and -elements

Pseudo-classes target states of an element (e.g., when a link is hovered or an input is focused), while pseudo-elements target specific parts of an element's content (e.g., first letter, before/after content).



# IMPORTANT CSS FEATURES

Here we will give examples of important CSS features you might most likely need in web application development. Of course there are many other features, see for instance [here](#).

# COLORS

```
<h1 style='color: magenta;'>What do you need color for?</h1>
<p style='color: #00ff00;'>for text ...</p>
<p style='background-color:#0000ff;
  color: rgb(255, 255, 255);'>
  ...but also for the background ...
</p>
```

- use properties "color" / "background-color" to set text color / background color
- color values can be predefined color names (see [here](#)), hex and RGB values.
- Color gradients are also possible, see e.g. [here](#)
- RGB and HEX syntax specify 3 color values from 0 - 255 for the three primary colors red, green and blue

## HEIGHT AND WIDTH

- properties "height" und "width" set height and width of an element
- Possible values: "auto" -> browser sets values based on content; or use values in units "px" or "%"
- "max-height"/"max-width": Maximum height or width of an element (analogue "min-height" etc.)
- other "overflow" with div3: "scroll" will show a scrollbar, "hidden" hides overflowing content

## BORDER, MARGIN UND PADDING

```
p {  
  border: 1px solid black;  
  margin: 10px 5px 10px 5px;  
  padding-left: 5px;  
  padding-right: 15px;  
  padding-top: 20px;  
  padding-bottom: 30px;  
}
```

- "border" defines the border width, the line type and the line color .
- "margin" defines the distance between an element and other elements.
- "padding" defines the distance between the border and the content of an element.
- All three properties can be set together (e.g. margin) or individually (e.g. padding).

## BOXMODELS

- A box model determines how the final size of an element is calculated.
- Use property "box-sizing" to set a box model: content-box (default) and border-box
- Width of content-box := content-width + padding (left + right) + border (left+right)
- Width border-box := width specification

## STYLING TEXTS

- Centred, left- or right-aligned arrangement of text with "text-align"
- "text-decoration": Display of a line at the top/bottom/center
- Indent the first line of a paragraph with "text-indent"
- Set the line height with "line-height"

## SETTING FONTS

- `font-family`: you specify three fonts, the first available one is used
- `font-style`: normal | italic
- `font-size`: character size in px, % or em
- `1em` corresponds to the current text size, 1em typically corresponds to 16px
- `font-weight`: normal / bold

## LINKS

- Pseudo classes :link, :visited, :hover, :active for specifying the appearance of unvisited, visited, "hovered" (mouse moves over link) or selected links
- Links can also be displayed as buttons, either inline (1st button) or as a block (2nd button)
- The values of the display property are explained later

## TABLES

- Designing the table borders with the border property, "border-collapse" avoids double lines
- Defining the width and height of the table or the table rows and columns (height, width)
- Alignment: Definition of the horizontal and vertical arrangement of the cell contents ("text-align")

*Tables can be styled as follows (continued):*

- Padding: Defining the distance between the margins and the text
- Table hover: Highlighting of rows when hovering with the mouse (`tr:hover`)
- Alternating background colour for rows with `nth-child(odd)` / `nth-child(even)`
- Responsive: Automatic scrollbar if the width or height of the browser window is not sufficient (`overflow-x`)

# ELEMENT ALIGNMENT

*Important CSS features for element alignment:*

- Center (horizontally/verically) an element by `margin: auto`
- Center a text by `text-align: center`
- Center an image (IMG tag): `margin: auto` and `display: block`
- Align to left oder right border by `float: left/right`
- Center text vertically by `padding` or `line-height`

## SETTING THE DISPLAY PROPERTY

Use `display` to determine whether an element is displayed and how it should be displayed.

- `none`: the element is not displayed and, unlike "visibility: hidden", is not included in the layout calculation
- `inline`: Display analogue to SPAN, "height"/"width" cannot be set
- `block`: Display analogue to P element



# RESPONSIVE WEB DESIGN

## Summary: "Tricks" for responsive web pages

- use a grid (typically 12 columns) → introduce points for line wrap
- use media query for small screens → redefine CSS rules
- hide/show components depending on display width
  - menu: show burger menu on small screens
  - table: hide table on small screens, show list of cards instead



## ADVANCED TOPICS

- Flexbox Tutorial: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- CSS Frameworks: [Bootstrap](#) (used in some examples in this course), [Tailwind](#) (used in ACME apps, see chapter 4 and 6)
- Advanced Stylesheets: SASS, see Tutorial on [https://www.w3schools.com/sass/sass\\_intro.php](https://www.w3schools.com/sass/sass_intro.php)



## EXERCISE: STYLING A FORM

A given HTML contact form shall now be turned into a better looking form by following the style guidelines on the next pages.

Unzip css-form.zip and open the folder with Visual Studio Code.

*With your CSS styling the form should look like this:*

## Contact Form

Name:

E-Mail:

Subject:

Text:

☐ Would you like to receive a copy of your email??

Send

*Please make the following adjustments:*

- Set the font for the entire document to Arial
- Labels should be at least `100px` wide and have a margin of `10px` downwards (*when can labels have widths?*)
- Text and mail input fields should be `300px` wide
- Input and textarea elements should be as large as the font size of the father, a padding to the left/right of `12px` and top/bottom of `6px` as well as a border with `2px` thickness in `lightgray` - round the border by `4px` with `border-radius`
- Textarea should be `100px` high and as wide as label and text input field together - make sure it is flush!

- If the cursor is in an input or textarea element, the background should have the colour `#EEE` (*which pseudo class is necessary here?*)
- The submit input should have the colour Font colour `White`, background colour `CornflowerBlue` and border colour `DarkBlue`. In addition, the width should be flush with the text area and the cursor should change to a pointer (`cursor: pointer`)
- Make sure that the submit input element changes the background colour to `DarkBlue` when the mouse is over the element.



# 1.4 JAVASCRIPT

# PREFACE

This section is not intended to give a full explanation of JavaScript (JS). Instead, it is assumed that the reader has some experience with JavaScript - accordingly we will give an overview of important JavaScript features by example.

Also we will take a short look at a JSON example.

Finally, the various possibilities of the JavaScript DOM API are demonstrated by a variety of example programmes.



# VARIABLES AND TYPES

```
var i = 1;           // number
const f = 1.2;       // number
var b = true;        // boolean
var s = 'Hello';     // string
let output = 'Type(i) = ' + typeof i + '<br>Type(f) = '
  + typeof f + '<br>Type(b) = ' + typeof b + '<br>Type(s) = '
  + typeof s;
document.writeln(output);
```

# OPERATORS

JS supports the usual operators for arithmetic operations and comparisons. However, other than statically typed languages such as Java, JS tries to match operands of different types. To avoid surprising or unwanted results, you should make sure that operands are compatible.

For comparisons it is highly recommended to use the "===" operator instead of "==" to avoid surprises. Visit the [Equality Table](#) if you are not sure why.

# CONTROL FLOW

JS supports very similar control flow statements for loops and conditional execution.

While "if-then-else" statements are identical, JS has some special "for loop" statements.

```
/* simple for loop with index */
document.writeln('index-based for-loop - numbers: ');
for (let i = 0; i < 5; i++)
    document.writeln(' ' + i);
/* iterate over object keys with for-in */
document.writeln('<br>Object keys with for-in loop:');
const obj = { name: 'Paula', age: 23 };
for (let key in obj)
    document.writeln(' ' + key);
/* iterate over array elements with for-of */
document.writeln('<br>Array enumeration with for-of loop:');
const numbers = [5, 3, 1];
for (let n of numbers)
    document.writeln(' ' + n);
```



# ARRAYS

```
const numbers = [3, -1, 10];
numbers[4] = 1000; // index access
numbers.push(100); // append 100
for (let i = 0; i < numbers.length; i++)
    document.writeln(' ' + numbers[i]);
numbers.sort(); // sort ascending
numbers.sort((n1, n2) => n2 - n1); // sort desc with arrow function
document.write('<br>');
numbers.forEach(n => document.write(' ' + n)); // loop alternative
document.write('<br>Positive numbers:');
numbers.filter(n => n > 0).forEach(n => document.write(' ' + n));
```



# OBJECTS

JS offers a type "object". An object is made up of a set of properties (the data of the object) and a set of functions.

The person object on the previous page can also be created in this way:

```
person = {
  firstName: 'Hugo',
  location: '',
  moveIn: function(loc) {
    this.location = loc;
  },
  toString: function() {
    return this.firstName + ' lives in ' + this.location;
  }
};
```

The above notation is very similar to the JavaScript Object Notation (JSON) discussed later. The difference is that functions may be defined here. This is NOT permitted with JSON.



# FUNCTIONS

Like many languages, JS offers a function concept. We distinguish between named functions and arrow functions.



# CLASSES AND INHERITANCE

JS offers a limited class concept compared to Java or TypeScript. It allows you to build an inheritance hierarchy based on single inheritance. There is no real access control as in "public", "private" or "protected". However, as of ECMAScript 2022 you can use "#attributes":



# ERROR HANDLING

JS supports error handling analogous to Java. You can use an Error class and derive from it, however, in many cases a try-catch construct is used where a simple string is thrown (instead of an error object):



# JAVASCRIPT OBJECT NOTATION (JSON)

JSON is very popular for transferring data over the Internet because it is readable and it can easily be processed, especially in JavaScript environments.

```
const personAsJSON = {  
  'name': 'Peter Maier',  
  'age': 25  
};  
const personAsString = JSON.stringify(personAsJSON);  
localStorage.person = personAsString; // store in localStorage  
const personLoaded = JSON.parse(localStorage.person); // load again  
document.write('<br>Person loaded from localStorage:<br>' + JSON.stringify(p
```

Keep in mind that - for obvious reasons - you cannot have functions in JSON objects.  
You can also have arrays of JSON objects using the `[]` brackets.

## EXERCISE: JSON EXERCISE

*What are the differences between JSON and JavaScript objects? Mark all correct statements.*

- ☐ JSON and JS objects are the same
- ☐ Values in JS objects may be of any valid type, in JSON only strings, numbers, arrays, true, false, null, or other JSON objects are possible.
- ☐ The property keys must be quoted (double quotes only) in JSON.

# JS DOM API

JS supports a very powerful API to access and manipulate the Document Object Model (DOM) in the browser. The API allows you to

- find HTML elements by id, name, tag name, class name and by CSS selector
- traverse through the document tree starting from the predefined "document" variable
- all nodes in the tree have properties such as "children", "nextSibling", "previousSibling" etc. to access tree elements
- manipulate the document elements by appending/inserting nodes, removing nodes and setting node attributes
- read and manipulate CSS attributes of HTML elements
- handle events caused by user interaction with the browser

Please refer to [JS DOM Intro](#) for an introduction.



# FINDING ELEMENTS AND CHANGING THEM



# DOM MANIPULATION

# LOADING DATA ASYNCHRONOUSLY

The standard model for web applications stipulates that web pages are sent from a web server to the browser and displayed there. Every significant interaction with the web page (form submit, navigation) leads to the web server being called again, which then delivers a new page. This request/response scheme is not suitable for interactive applications, as each page change is accompanied by a delay that impairs the user experience.

In JavaScript developers have a choice of two APIs to solve the problem: AJAX and the fetch-API.

## AJAX (ASYNCHRONOUS JAVASCRIPT AND XML) APPROACH

1. The browser sends an asynchronous call to an HTTP server
2. The server responds with an XML or JSON message
3. The content of the message is written to the DOM and displayed immediately

As a result

- the undesired page reload is omitted
- loaded data is almost immediately visible in the browser
- or data seems to be immediately saved on the server



## EXAMPLES FOR AJAX, PROMISES AND FETCH

Unzip ajax-demo.zip and open the folder with Visual Studio Code.

First we will take a close look at "ajax-demo.html". It demonstrates how you can load JSON data asynchronously from a server and display the data in the browser.

Then we will introduce Promises which let you register a handler method for data that arrive at a later point in time. An example can be found in "promise-demo.html".

Finally we will take a look at "fetch-demo.html" to see how we can use Promises together with the new fetch API to load and process data.

# AJAX DEMO

```
<!DOCTYPE html>
<html>
<body>
  <div id="server-text">server reply will be here...</div>
  <script>
    ... JS code, see next page
  </script>
</body>
</html>
```

- the JS script will setup an AJAX call to a server to load and process data
- the JS code will store the data in a DIV with id="server-text"

## JS code to load and process JSON data from a server

```
var xhr = new XMLHttpRequest();
xhr.onreadystatechange = function () {
    if (xhr.readyState == 4) {
        if (xhr.status == 200) {
            const div = document.getElementById("server-text");
            div.innerHTML = xhr.responseText;
        } else {
            console.error("Error (" + xhr.status + ")");
        }
    }
};
xhr.open("GET", "https://online-lectures-cs.thi.de/chat/test.txt", true);
xhr.send();
```

- create a XMLHttpRequest object, this is the actual AJAX API
- first register a handler function which gets called when communication state changes
- when data arrive (readyState == 4), we check for errors and store the data
- setup and send the request

# PROMISES DEMO

A Promise in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value. It acts as a placeholder for a value that may not be available yet but will be resolved (or rejected) at some point in the future.

Typical usage scenario:

## 1. create the promise

```
const promise = new Promise(<promise function>);
```

## 2. register your handler functions for result and error processing

```
promise.then(result => {  
    // result processing...  
})  
.catch(error => {  
    // error processing...  
});
```

## Promises Part I - Creating a Promise

```
const getPerson = (resolve, reject) => {  
  const n = Math.floor(Math.random(1)*100);  
  
  if (n > 50) {  
    const name = "Name" + n;  
    const age = n-40;  
    resolve('{ "name": "' + name + '", "age": ' + age + ' }');  
  } else {  
    // error!  
    reject(new Error('no data!'));  
  }  
};  
const promise = new Promise(getPerson);
```

- a Promise object is created. Function "getPerson" is executed and randomly returns either a person or an error object
- resolve returns a person object in JSON format, which can be processed with "then" (see next page)
- reject produces an error object that can be handled with "catch"

## Promises Part II - Processing results or errors

```
promise
.then((jsonText) => {
  return JSON.parse(jsonText);
})
.then((person) => {
  if (person.age > 40) {
    // too old - throw exception
    throw new Error("This person is too old, age = " + person.age);
  }
  console.log("name = " + person.name + ", age = " + person.age);
})
.catch((error) => {
  // Handle errors if age is too high
  console.error('Error: ' + error.message);
});
```

- Step 1: Convert JSON text into person object, the return value is processed by the next "then"
- Step 2a: Check person object, an error is handled in "catch"
- Step 2b: Output person object
- Error handling: Output error - either from "reject" or from "throw" in previous "then"

# FETCH DEMO

Similar to the AJAX demo we have a HTML scaffold that offers an empty output tag to store the data after they arrive:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Fetch Demo</title>
  </head>
  <body>
    <h3>Fetch Demo</h3>
    <p>Fetching data from ...</p>
    <output id="result">nothing yet</output>
    <script>
      ... JS part see next page
    </script>
  </body>
</html>
```

## JS code to load and process JSON data from a server

```
fetch("https://jsonplaceholder.typicode.com/posts")
.then(function (response) {
  if (response.ok) {
    return response.json();
  } else {
    throw new Error("HTTP error " + response.status);
  }
})
.then(function (data) {
  let personsAsString = '';
  data.forEach((person) => {
    personsAsString += `${person.id}: ${person.title}<br>`;
  });
  document.getElementById('result').innerHTML = personsAsString;
})
.catch(function (error) {
  document.getElementById('result').innerHTML = error;
});
```

- fetch creates a HTTP GET request and returns a promise
- first step: transform response body to JSON format
- second step: format persons into text and store in output tag
- error handling



General remark: the examples shown just give an idea of how AJAX or fetch calls can be performed.

For more information check out for example the following resources:

- AJAX: see [w3schools/ajax intro](#)
- Fetch: see [MSDN Docs on fetch](#)
  - here you can also see different, i.e. shorter approaches using "await"

# 1.5 EXERCISE: TODO-APP

Develop a simple Web application for managing a todo list. It works as follows:



The screenshot shows a web application titled "My Todo List". It features a form for adding new todos with a text input field labeled "new todo:" containing the placeholder text "describe todo..." and an "add" button. Below the form, there is a section titled "Current Todos" which displays a list of three items: "go shopping today", "clean room", and "program a cool app". Each item has a green "done" button next to it.

App features:

- list open todos (todos with state "open") or display message
- close a todo (it will receive state "closed")
- create a new todo (empty input validation will mark input field red)

